YENEPOYA INSTITUTE OF TECHNOLOGY

NBA-Accredited : B.E (CSE,IOT,BT)

(Affiliated to Visvesvaraya Technological University, Belagavi)

N.H. 13, Thodar, Moodbidri, Mangaluru 574225 (D.K), Karnataka

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(IOT,Cyber Security including Block Chain Technology)

LAB MANUAL

Blockchain Technology

BIC702

[As per Choice Based Credit System (CBCS) scheme]

Vision

To empower students such that they will be technologically adept, innovation driven, self-motivated and responsible global citizens possessing human values by imparting quality technical education in the field of Computer Science.

Mission

1. To Facilitating and exposing the students to various learning opportunities through dedicated academic guidance and monitoring.
2. To Provide a learning ambience to encourage innovations, problem solving skills, leadership qualities, team-spirit, entrepreneurship skills and ethical responsibilities.
3. To Encourage faculty and students to actively participate in innovation, industry solutions,research and lifelong learning so that their contribution makes a substantial difference to the society.

Program Educational Objectives (PEO's):

- Graduates will be equipped to be employed in IT industries and be engaged in learning understanding and applying new ideas.
- Graduates through academic training will be able to take up higher studies and industry specific research programs.
- Graduates will be responsible computing professional with social obligations in their own area of interest.

Program Specific Outcomes (PSO's):

- Graduates will be able to use the knowledge and ability to write programs and integrate them with the hardware/software products in the domains of embedded systems, databases/data analytics, network/web systems and mobile products.
- Graduate will be able to use knowledge of information science in various domains to identify research
- gaps and hence to provide solution to new ideas and innovations.

Program Outcomes

1. **Engineering knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. **Problem analysis**: Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. **Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write

effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. **Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long le

13. arning in the broadest context of technological change.

BIC702–Blockchain Technology Laboratory Course Outcomes	
BIC702 .1	Understand the blockchain Technologies with its applications.
BIC702 .2	Examine and apply the decentralization,asymmetric,cryptographic,primitives and Bitcoin concepts.
BIC702.3	Analyse the principles of Ethereum and its transactions in Blockchain.
BIC702.4	Apply ethical principles and demonstrate a private blockchain using ethereum Tool.

TABLE OF CONTENT

SL. NO.	PARTICULARS	PAGE NO.
1	**Write a program to generate public and private key using OpenSSL.**	1-2
2	**Write program to create a simple Blockchain using Python.**	3-7
3	**Develop and test smart contract on Ethereum test networks.**	8-13
4	**Develop a program to implement blockchain in Merkle Trees.**	14-17
5	**Design and develop Cryptocurrency for multiple user using python.**	18-20
6	**Write and deploy chain code in Hyperledger Fabric.**	21-22
7	**Write a smart contract using a solidity program to perform the balantransfer from contract to other accounts.**	23-26
8	**Write a program to perform token Creation and Management on Ethereum**	27-30
9	**Setup Metamask in the system and create a wallet in the Metamask with TestNetwork**	31-34
10	**Develop a program to implement blockchain in Merkle Trees.**	35-37
11	**Create multiple accounts in metamask and perform the balance transfer between the accounts and describe the transaction specification**	38-41
12	**Setup the Hyperledger Fabric Network with 2 Organizations 1 Peer Each in the system**	42

(Effective from the academic year 2025)SEMESTER VII

Course Code	BIC702	CIE Marks	50
Number of Contact Hours/Week	0:0:2	SEE Marks	50
Total Number of Lab Contact Hours	20	Theory Exam Hours	03

Course objectives:

- To Demonstrate the need for OS and different types of OS
- To discuss suitable techniques for management of different resources
- To demonstrate different APIs/Commands related to processor, memory, storage and file systemmanagement.

Descriptions (if any):

- Simulation packages preferred: Multisim, Xilinx _ISE9.2, PSpice or any other relevant.
- For Analog Electronic Circuits students must trace the wave form on Tracing sheet /Graph sheetand label trace.
- Continuous evaluation by the faculty must be carried by including performance of a studentinboth hardware implementation and simulation (if any) for the given circuit.
- A batch not exceeding 4 must be formed for conducting the experiment. For simulationindividual student must execute the program.

Experiment 1

Write a program to generate public and private key using OpenSSL.

Aim

To use OpenSSL to create a secure 2048-bit RSA private key (with password protection) and generate its matching public key.

Procedure

Step 1: Install OpenSSL

```
sudo apt install openssl
```

Step 2: Create a bash script file named generate_keys.sh

```
cat > generate_keys.sh
```

Then paste the following code inside:

```
#!/bi
n/bas
h set
-e
PRIVATE_KEY="private_key.pem"
PUBLIC_KEY="public_key.pem"

openssl genpkey -algorithm RSA -out "$PRIVATE_KEY" -aes-256-cbc
-pkeyopt rsa_keygen_bits:2048 echo "Private key saved to
$PRIVATE_KEY."

openssl pkey -in "$PRIVATE_KEY" -pubout -out
"$PUBLIC_KEY"s echo "Public key saved to
$PUBLIC_KEY." Press CTRL+D to save and exit.
```

Step 3: Make the script executable chmod +x generate_keys.sh

Step 4: Run the script
./generate_keys.sh

Step 5: Display the generated keys cat private_key.pem cat public_key.pem

Conclusion

OpenSSL was used to create a 2048-bit RSA private key (password protected) and its public key, both shown in PEM format.

Experiment 2: Write program to create a simple Blockchain using Python.

Code:

```
#!/usr/bin/env python3
"""
Minimal Blockchain in Python
- Blocks store index, timestamp, transactions, previous_hash, nonce, and hash
- Simple Proof-of-Work (PoW): find a nonce so the block hash starts with N zeros
- Pending transactions are included when mining a new block
"""

from __future__import annotations
from dataclasses import dataclass, field, asdict
from typing import List, Any
import hashlib
import json
import time


def sha256_hexdigest(data: Any) -> str:
    """Stable SHA-256 of a Python object via sorted JSON encoding."""
    encoded = json.dumps(data, sort_keys=True, separators=(",", ":")).encode("utf-
8")
    return hashlib.sha256(encoded).hexdigest()


@dataclass
class Block:
    index: int
    timestamp: float
    transactions: List[dict]
    previous_hash: str
    nonce: int = 0
    hash: str = field(init=False, default="")

    def compute_hash(self) -> str:
        # Exclude the 'hash' field itself from the digest input
        block_dict = asdict(self).copy()
```

```
        block_dict.pop("hash", None)
        return sha256_hexdigest(block_dict)

    def mine(self, difficulty: int) -> None:
        """Proof-of-Work: find a nonce so that hash has leading 'difficulty' zeros."""
        assert difficulty >= 0
        prefix = "0" * difficulty
        self.nonce = 0
        while True:
            self.hash = self.compute_hash()
            if self.hash.startswith(prefix):
                break
            self.nonce += 1


class Blockchain:
    def __init__(self, difficulty: int = 3, mining_reward: int = 50):
        self.difficulty = difficulty
        self.mining_reward = mining_reward
        self.chain: List[Block] = [self._create_genesis_block()]
        self.pending_transactions: List[dict] = []

    def _create_genesis_block(self) -> Block:
        block = Block(
            index=0,
            timestamp=time.time(),
            transactions=[{"system": "genesis"}],
            previous_hash="0" * 64,
        )
        block.mine(self.difficulty)
        return block

    @property
    def last_block(self) -> Block:
        return self.chain[-1]

    def add_transaction(self, sender: str, recipient: str, amount: float) -> None:
        if amount <= 0:


            raise ValueError("Amount must be positive")
```

```
        tx = {"sender": sender, "recipient": recipient, "amount": float(amount)}
        self.pending_transactions.append(tx)

    def mine_pending_transactions(self, miner_address: str) -> Block:
        """Create a new block from pending txs, add a mining reward, mine it, and
append."""
        # Include reward as a transaction from 'SYSTEM'
        reward_tx = {"sender": "SYSTEM", "recipient": miner_address, "amount":
self.mining_reward}
        transactions = self.pending_transactions + [reward_tx]

        new_block = Block(
            index=len(self.chain),
            timestamp=time.time(),
            transactions=transactions,
            previous_hash=self.last_block.hash,
        )
        new_block.mine(self.difficulty)
        self.chain.append(new_block)

        # Reset pending transactions
        self.pending_transactions = []
        return new_block

    def is_chain_valid(self) -> bool:
        """Validate hashes, PoW, and linkage across the entire chain."""
        if not self.chain:
            return False

        # Validate genesis separately
        genesis = self.chain[0]
        if genesis.hash != genesis.compute_hash():
            return False
        if not genesis.hash.startswith("0" * self.difficulty):
            return False

        # Validate subsequent blocks
```

```
        for i in range(1, len(self.chain)):
            curr = self.chain[i]
            prev = self.chain[i - 1]

            # Recompute and check PoW
            if curr.hash != curr.compute_hash():
                return False
            if not curr.hash.startswith("0" * self.difficulty):
                return False
            # Check linkage
            if curr.previous_hash != prev.hash:
                return False

        return True

    def to_dict(self) -> dict:
        """Serialize the whole chain to a JSON-friendly dict."""
        return {
            "difficulty": self.difficulty,
            "mining_reward": self.mining_reward,
            "length": len(self.chain),
            "chain": [
                {
                    "index": b.index,
                    "timestamp": b.timestamp,
                    "transactions": b.transactions,
                    "previous_hash": b.previous_hash,
                    "nonce": b.nonce,
                    "hash": b.hash,
                }
                for b in self.chain
            ],
        }


def demo():
    print("Creating blockchain with difficulty=3 ...")
    bc = Blockchain(difficulty=3, mining_reward=25)
```

```
    print("Adding transactions...")
    bc.add_transaction("Alice", "Bob", 10)
    bc.add_transaction("Bob", "Charlie", 4.25)

    print("Mining pending transactions for Miner1 ...")
    block1 = bc.mine_pending_transactions("Miner1")
    print(f"Block #{block1.index} mined: {block1.hash}")

    print("Adding more transactions...")
    bc.add_transaction("Alice", "Dora", 2.5)
    bc.add_transaction("Charlie", "Alice", 1.2)

    print("Mining pending transactions for Miner1 again ...")
    block2 = bc.mine_pending_transactions("Miner1")
    print(f"Block #{block2.index} mined: {block2.hash}")

    print("\nChain valid?", bc.is_chain_valid())

    print("\nFull chain (JSON):")
    print(json.dumps(bc.to_dict(), indent=2))


if __name__ == "__main__":
    demo()
```

Experiment 3

Develop and test smart contract on local Blockchain.

Aim

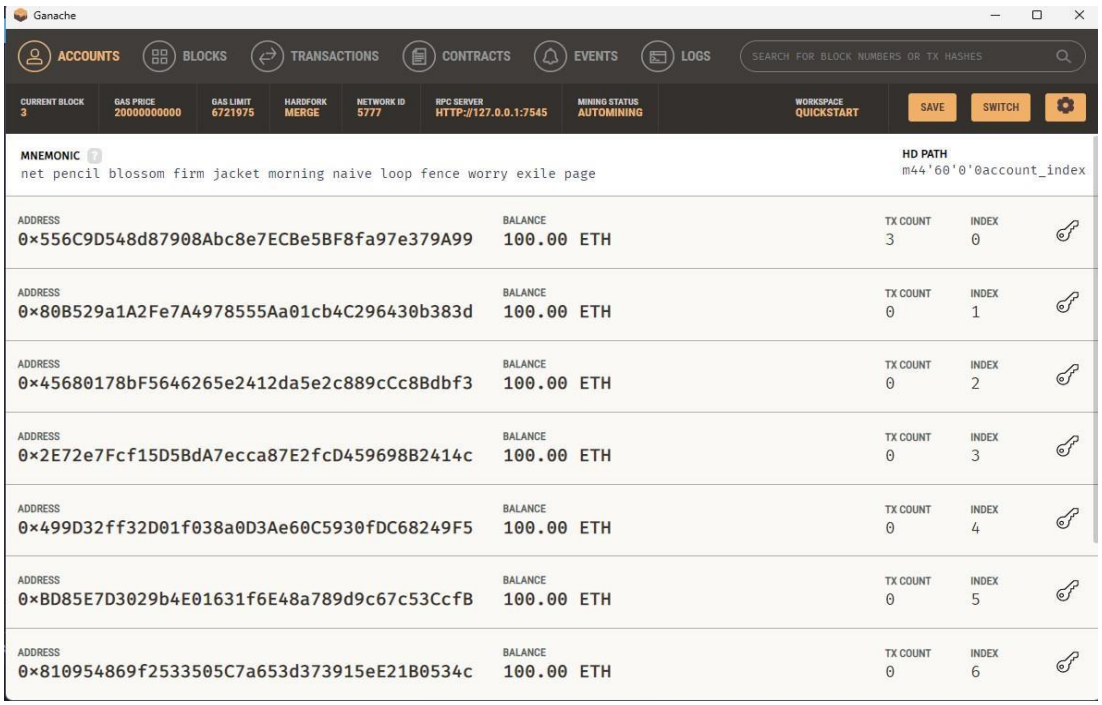
To develop, deploy, and test a simple smart contract (Storage) on a local blockchain using Ganache, MetaMask, and Remix IDE.

Procedure

Step 1: Start Ganache

1. Open Ganache
2. Click Quickstart Ethereum
3. Keep Ganache running for the whole experiment.

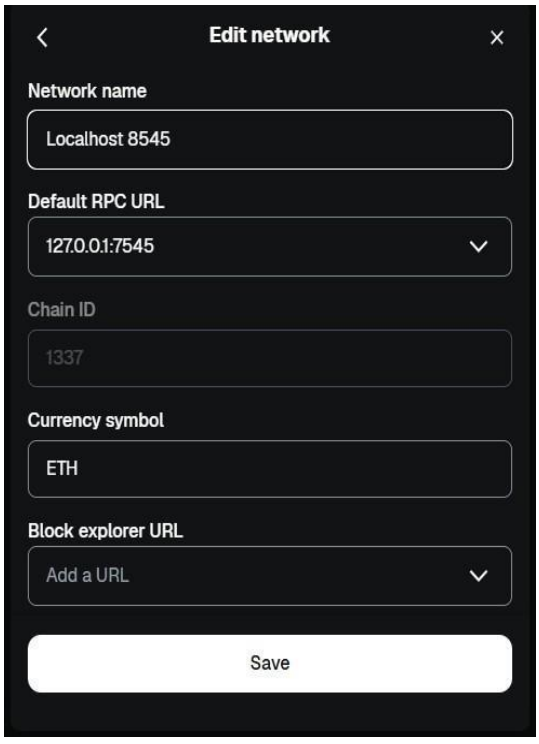
Output:



Step 2: Connect Ganache to MetaMask

- Open MetaMask → Settings → Networks → Add network manually
- Enter:
  - Network Name: Localhost 8545
  - RPC URL: http://127.0.0.1:7545
  - Chain ID: 1337
  - Currency: ETH
- Save network

Output:



Step 3: Import Ganache Account into MetaMask

1. In Ganache, copy a private key (click key icon)
2. MetaMask → Account → Import Account
3. Paste private key → Import

Step 4: Open Remix IDE

- Visit https://remix.ethereum.org
- **Create a new file named** Storage.sol
- **Paste the following code:**
- // SPDX-License-Identifier: GPL-3.0 pragma solidity ^0.6.0;
- contract Storage { uint256 number;

```
    function store(uint256 num) public {
number = num;
    }
    function retrieve() public view returns (uint256)
{       return number;
    }
}
```

Step 5: Compile Contract

- Click Solidity Compiler
- Select Compiler Version 0.6.0
- Click Compile Storage.sol

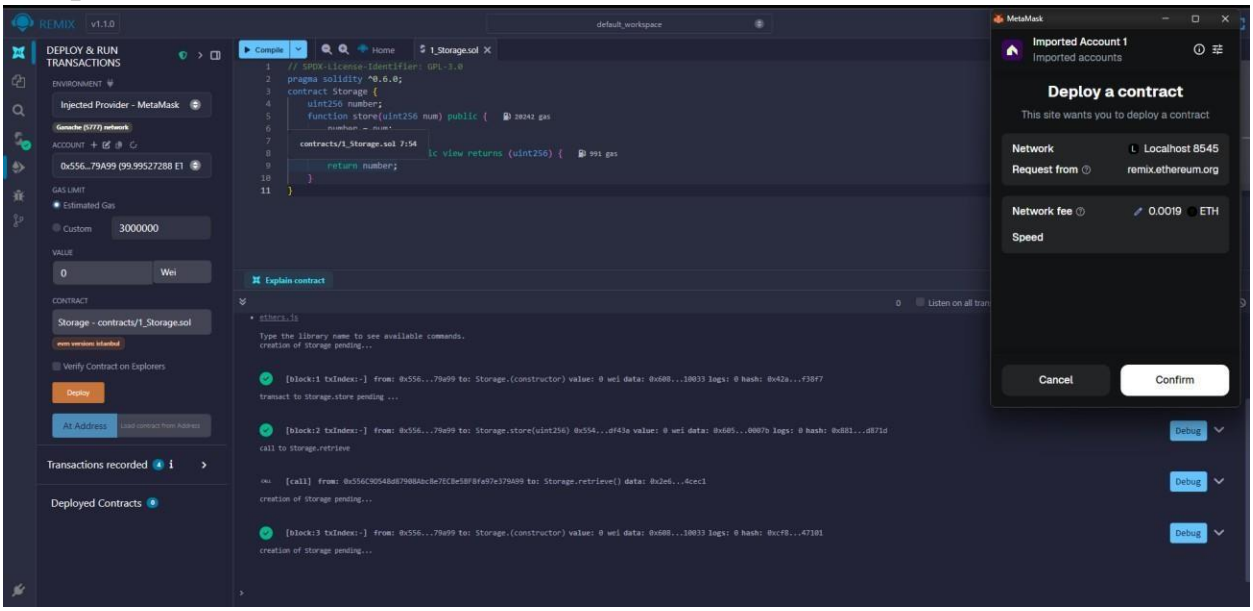
Step 6: Connect Remix to MetaMask

- Click Deploy & Run Transactions
- Environment → select Injected Provider - MetaMask
- MetaMask will connect to Ganache

Step 7: Deploy Contract to Local Blockchain

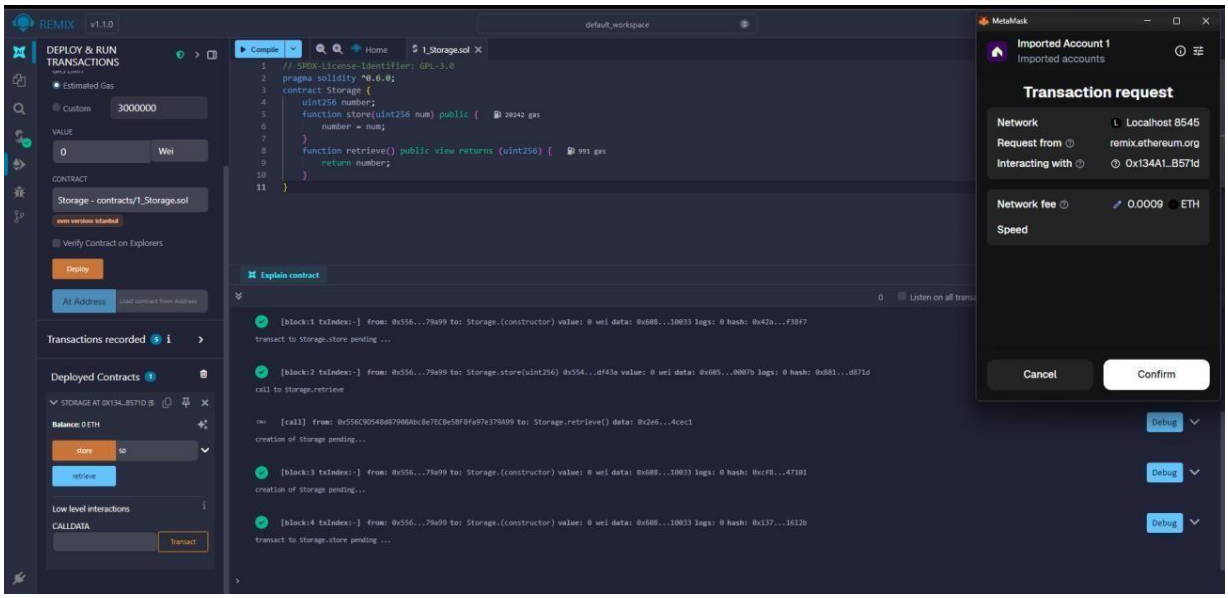
- Click Deploy
- Accept the transaction in MetaMask

Output:



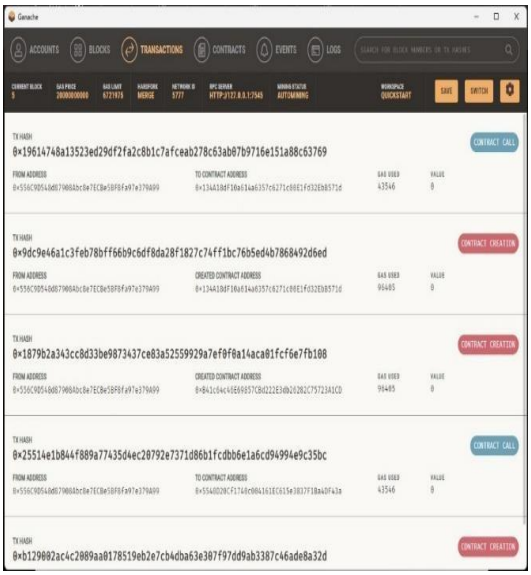
Step 8: Interact With Contract

- Under Deployed Contracts o Click retrieve() → should show 0 o Enter a number in store() (example 50) → click button → confirm in MetaMask o Click retrieve() again → now shows 50



Step 9: Verify Transaction in Ganache

- Go to Ganache → Transactions tab
- Confirm your contract deployment and function transactions



Conclusion

The smart contract was successfully developed, deployed, and tested on a local blockchain using Ganache, MetaMask, and Remix, demonstrating basic Ethereum smart contract functionality and interaction.

Experiment 4

Develop and test smart contract on Ethereum test networks.

Aim

To deploy a simple smart contract (SimpleStorage) on the Sepolia Ethereum Testnet using Remix IDE and interact with it.

Requirements

- MetaMask browser extension (installed & set to Sepolia Testnet)
- Sepolia test ETH from faucet: Google Cloud Faucet
- Remix IDE: Remix IDE

Procedure

Step 1: Set up MetaMask

1. Install MetaMask extension in your browser.
2. Switch to Sepolia Test Network in MetaMask.
3. Get test ETH from Google Cloud Faucet.

Step 2: Create the Smart Contract in Remix

1. Open Remix IDE: Remix IDE
2. In the File Explorer, go to the contracts folder.
3. Click New File and name it SimpleStorage.sol.
4. Paste the following Solidity code:

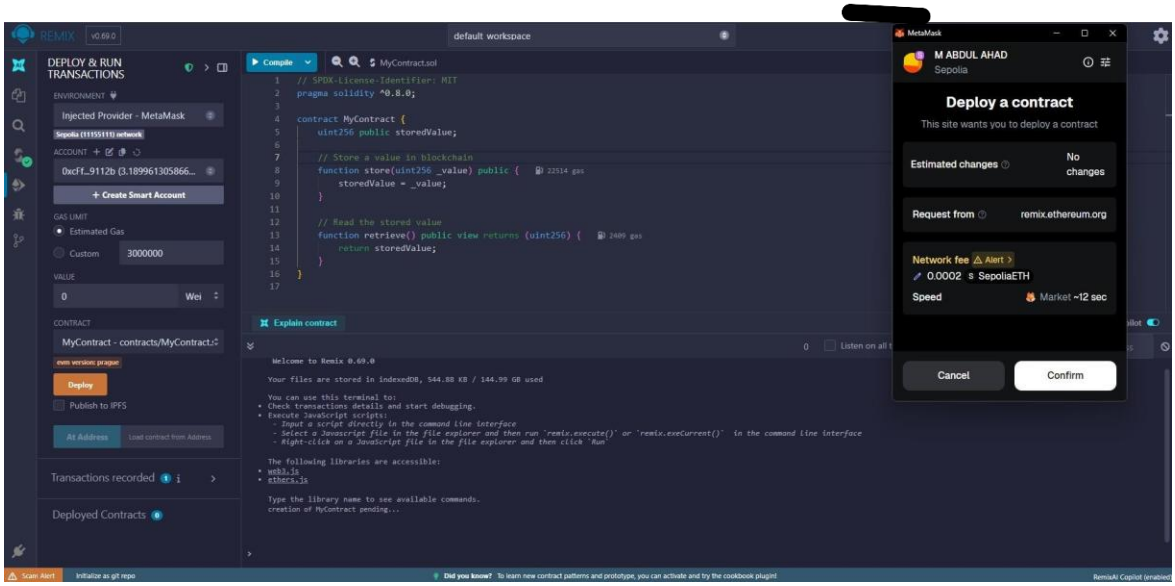
```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.0;
contract SimpleStorage {
uint256 public storedValue;
        function store(uint256 _value) public
    {       storedValue = _value;
```

```
    }

    function retrieve() public view
returns (uint256) {        return
storedValue;
    }
}
```

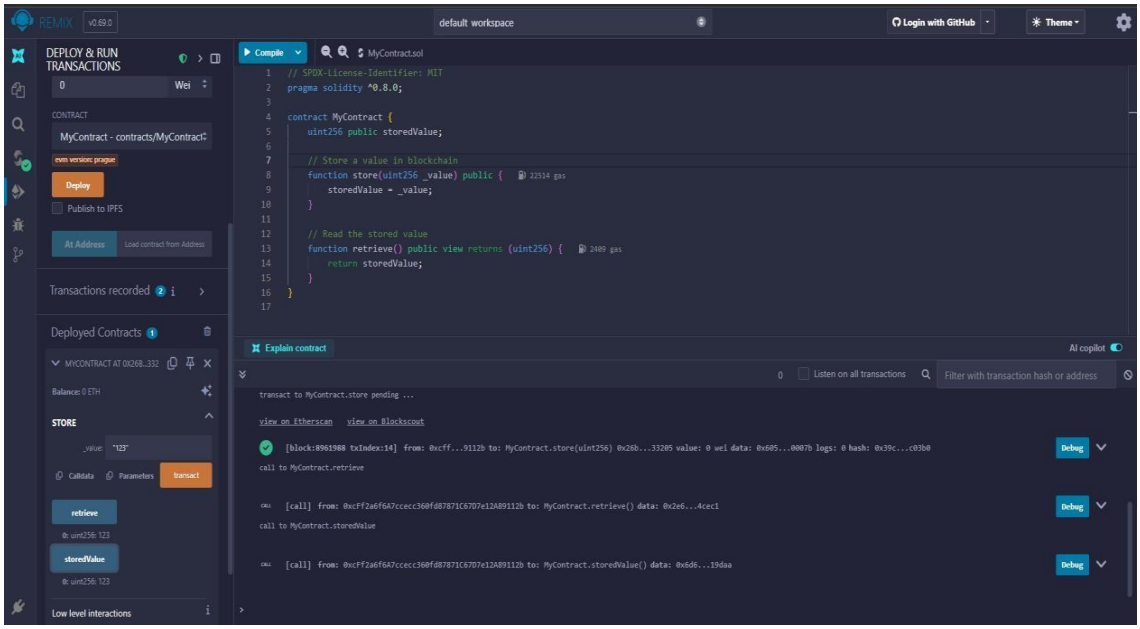
Step 3: Compile the Contract

1. Click the Solidity Compiler icon in the left menu.
2. Select compiler version 0.8.x.
3. Click Compile SimpleStorage.sol.



Step 4: Connect Remix to MetaMask

1. Go to Deploy & Run Transactions tab.
2. Under Environment, select Injected Web3.
3. Allow MetaMask to connect to Remix.

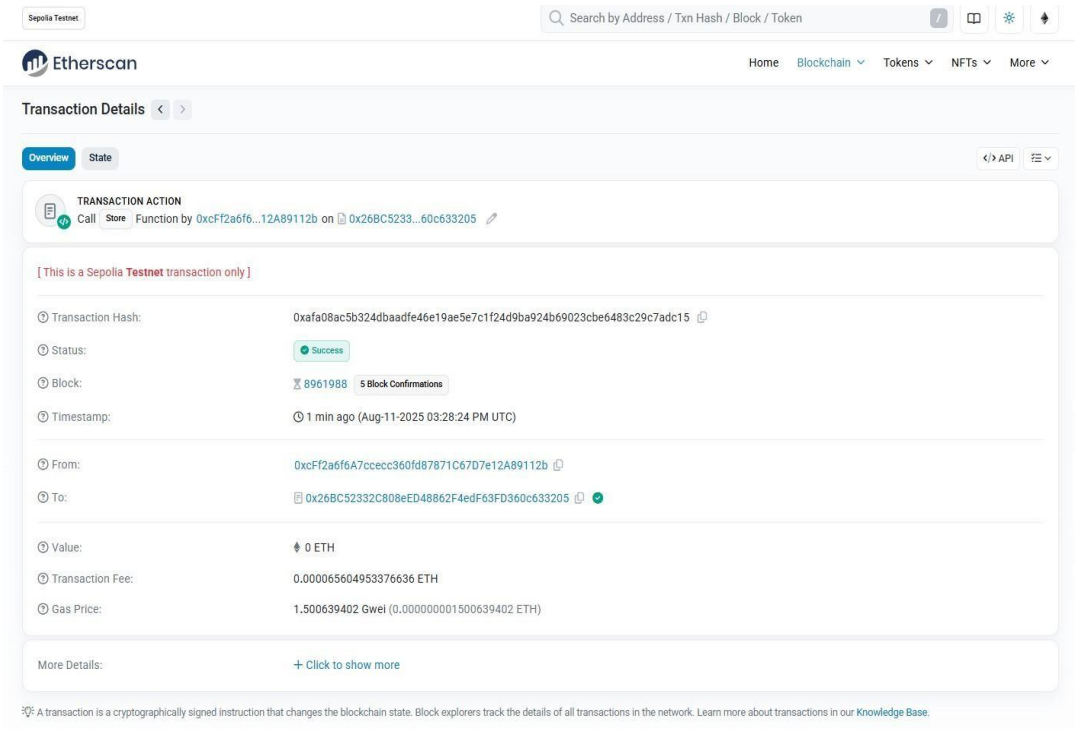


Step 5: Deploy the Contract

1. Make sure SimpleStorage is selected under Contract.
2. Click Deploy.
3. Confirm the transaction in MetaMask.
4. Wait for the deployment to complete.

Step 6: Interact with the Contract

1. Expand the deployed contract in Remix.
2. Enter a value (e.g., 123) in the store function and click store.
3. Confirm the transaction in MetaMask.
4. Click retrieve to see the stored value.



Conclusion:

We deployed and tested a smart contract on the Sepolia Testnet using Remix and MetaMask, gaining hands-on experience in blockchain interaction.

Experiment 5

Design and develop Cryptocurrency for multiple user using python.

Aim

To implement a simple cryptocurrency system in Python that allows users to create accounts, make transactions, and store them in a blockchain.

Procedure

Step 1: Open VS Code

- Launch Visual Studio Code on your computer.

Step 2: Create a New Python File

- Click File → New File.
- Save it as mini_crypto.py (use .py extension).

Step 3: Write the Code

- Copy and paste the following Python code into mini_crypto.py:

```
import time, hashlib, json
class Block:
def __init__(self, index, transactions, prev_hash):
self.index = index

self.time = time.strftime("%Y-%m-%d %H:%M:%S")

self.transactions = transactions

 self.prev_hash =prev_hash

self.hash = hashlib.sha256(json.dumps({"index": index,
"time": self.time,"transactions": transactions,
"prev_hash": prev_hash},
sort_keys=True).encode()).hexdigest()

class Blockchain:
def __init__(self):
self.chain = [Block(0, ["Genesis Block"], "0")]
self.pending = []
self.users = {}
def add_user(self, name, balance=100):
if name in self.users:
return "User already exists" self.users[name] = balance
return f"User {name} added with balance {balance}"
def add_transaction(self, sender, receiver, amount):
```

```
if sender not in self.users or receiver not in self.users:
return "Both users must exist"   amount = int(amount)
if self.users[sender] < amount:  return "Insufficient balance"
self.pending.append({"sender": sender, "receiver": receiver, "amount": amount})
return "Transaction addeddef mine(self):
if not self.pending:
return "No transactions to mine"
for tx in self.pending:
self.users[tx["sender"]] -= tx["amount"]
self.users[tx["receiver"]] += tx["amount"]
block = Block(len(self.chain), self.pending, self.chain[-1].hash)
self.chain.append(block)  self.pending = []
return f"Block {block.index} mined successfully"
def show_chain(self):
for block in self.chain:
print(f"\nBlock {block.index} [{block.time}]")
for tx in block.transactions:
print(" ", tx)

bc = Blockchain()
while True:
print("\n1. Add User  2. Add Transaction  3. Mine  4. Show Chain  5. Show Users  0. Exit")
choice = input("Choice: ")
if choice == "1":
print(bc.add_user(input("Enter name: ")))
elif choice == "2":
print(bc.add_transaction(input("From: "), input("To: "), input("Amount: ")))
elif choice == "3":
print(bc.mine())
elif choice == "4":
bc.show_chain()
elif choice == "5":
print(bc.users)
elif choice == "0":
break
```

Step 4: Save the File

- Press Ctrl + S (or File → Save).

Step 5: Run the Code

- Open Terminal in VS Code (shortcut: Ctrl + `).
- Run the program using:

  python mini_crypto.py

Output:

```
PS D:\files\blockchain> python mini_crypto.py

1. Add User  2. Add Transaction  3. Mine  4. Show Chain  5. Show Users  0. Exit
Choice: 1
Enter name: Alice
User Alice added with balance 100

1. Add User  2. Add Transaction  3. Mine  4. Show Chain  5. Show Users  0. Exit
Choice: 1
Enter name: Bob
User Bob added with balance 100

1. Add User  2. Add Transaction  3. Mine  4. Show Chain  5. Show Users  0. Exit
Choice: 1
Enter name: Charlie
User Charlie added with balance 100

1. Add User  2. Add Transaction  3. Mine  4. Show Chain  5. Show Users  0. Exit
Choice: 5
{'Alice': 100, 'Bob': 100, 'Charlie': 100}

1. Add User  2. Add Transaction  3. Mine  4. Show Chain  5. Show Users  0. Exit
Choice: 2
From: Alice
To: Bob
Amount: 10
Transaction added
```

```
1. Add User  2. Add Transaction  3. Mine  4. Show Chain  5. Show Users  0. Exit
Choice: 2
From: Charlie
To: Alice
Amount: 9999
Insufficient balance

1. Add User  2. Add Transaction  3. Mine  4. Show Chain  5. Show Users  0. Exit
Choice: 3
Block 1 mined successfully

1. Add User  2. Add Transaction  3. Mine  4. Show Chain  5. Show Users  0. Exit
Choice: 4

Block 0 [2025-08-16 17:34:18]
  Genesis Block

Block 1 [2025-08-16 17:36:51]
  {'sender': 'Alice', 'receiver': 'Bob', 'amount': 10}

1. Add User  2. Add Transaction  3. Mine  4. Show Chain  5. Show Users  0. Exit
Choice: 5
{'Alice': 90, 'Bob': 110, 'Charlie': 100}

1. Add User  2. Add Transaction  3. Mine  4. Show Chain  5. Show Users  0. Exit
Choice: 0
PS D:\files\blockchain>
```

Conclusion

This experiment shows how blockchain works in a simple way. We created users, made transactions, mined them into blocks, and linked the blocks with hashes to form a secure chain.

Experiment 6 :Write and deploy chain code in Hyperledger Fabric

Aim:

To set up the Hyperledger Fabric environment and start a test network in order to prepare for deploying and testing chaincode for asset management operations.

Explanation:

Hyperledger Fabric is a permissioned (private) blockchain platform that allows organizations to create their own blockchain network with controlled access. It supports smart contracts (called chaincode) to manage assets and perform transactions.

Procedure:

Step 1: Enable Windows Subsystem for Linux (WSL2)

Open PowerShell as Admin and run:

dism.exe/online/enable-feature /featurename:Microsoft-Windows- Subsystem-Linux /all /norestart

Step 2: Enable Virtual Machine Platform

dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart

Step 3: Set WSL2 as Default

wsl --set-default-version 2

Step 4: Install Ubuntu 20.04 LTS

Open Microsoft Store, search Ubuntu 20.04 LTS, and install it.

Step 5: Update Ubuntu & Install Packages

sudo apt update && sudo apt upgrade -y sudo

apt install nodejs npm -y sudo apt install curl

git -y

Step 6: Install Docker

- Download and install Docker Desktop for Windows
- Enable WSL2 integration for Ubuntu-20.04 inside Docker Desktop in Settings → Resources → WSL Integration.
- Verify installation: docker --version docker-compose --version

Step 7: Download Hyperledger Fabric Samples & Binaries

mkdir -p $HOME/go/src/github.com/ cd $HOME/go/src/github.com/ sudo curl -

sSL https://bit.ly/2ysbOFE | bash -s

Step 8: Start Fabric Test Network

cd fabric-samples/test-network

./network.sh up

Conclusion:

The Hyperledger Fabric test network was successfully started, and the environment is now ready for deploying chaincode, creating channels, and performing asset management operations in future experiments.

Experiment 7

Write a smart contract using a solidity program to perform the balantransfer from contract to other accounts.

Aim

To write, deploy, and test a Solidity smart contract that accepts Ether deposits, displays the contract balance, and transfers Ether (in wei) from the contract to another account.

Procedure

Step 1: Open Remix IDE

- Go to Remix IDE
- Create a new file → name it BalanceTransfer.sol

Step 2: Write the Smart Contract Code

Paste the following Solidity code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract BalanceTransfer { function
deposit() public payable {}

function getBalance() public view returns (uint) {
return address(this).balance;
}
function transferEther(address payable _to, uint _amount) public {
require(_amount <= address(this).balance, "Not enough balance");
(bool sent, ) = _to.call{value: _amount}("");
require(sent, "Transfer failed");
}
}
```

Step 3: Compile the Contract

- Go to Solidity Compiler in Remix.
- Select version 0.8.x.
- Click Compile BalanceTransfer.sol

Step 4: Deploy the Contract

- Go to Deploy & Run Transactions.
- Environment → Injected Provider (MetaMask).
- Click Deploy and confirm in MetaMask.

Step 5: Deposit 2 wei into the Contract

- In the Value field, enter 2 wei.
- Call the deposit() function.
- Confirm the transaction in MetaMask.

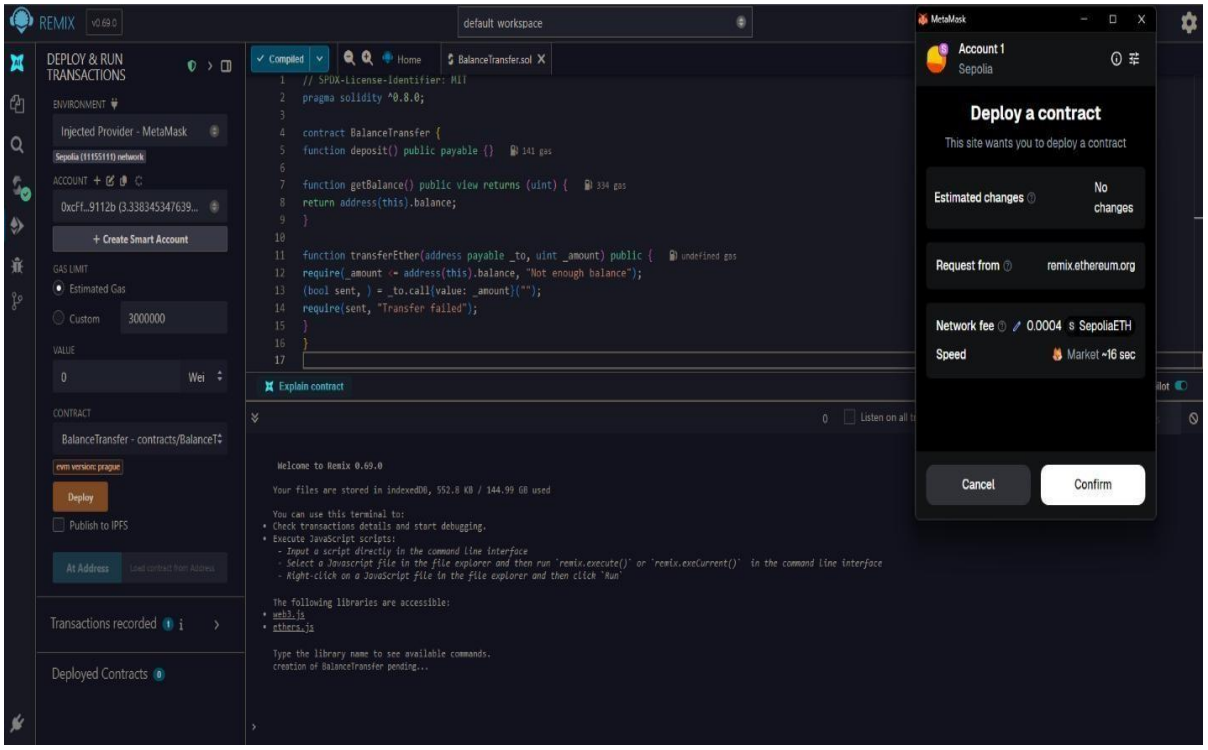
Step 6: Check Balance

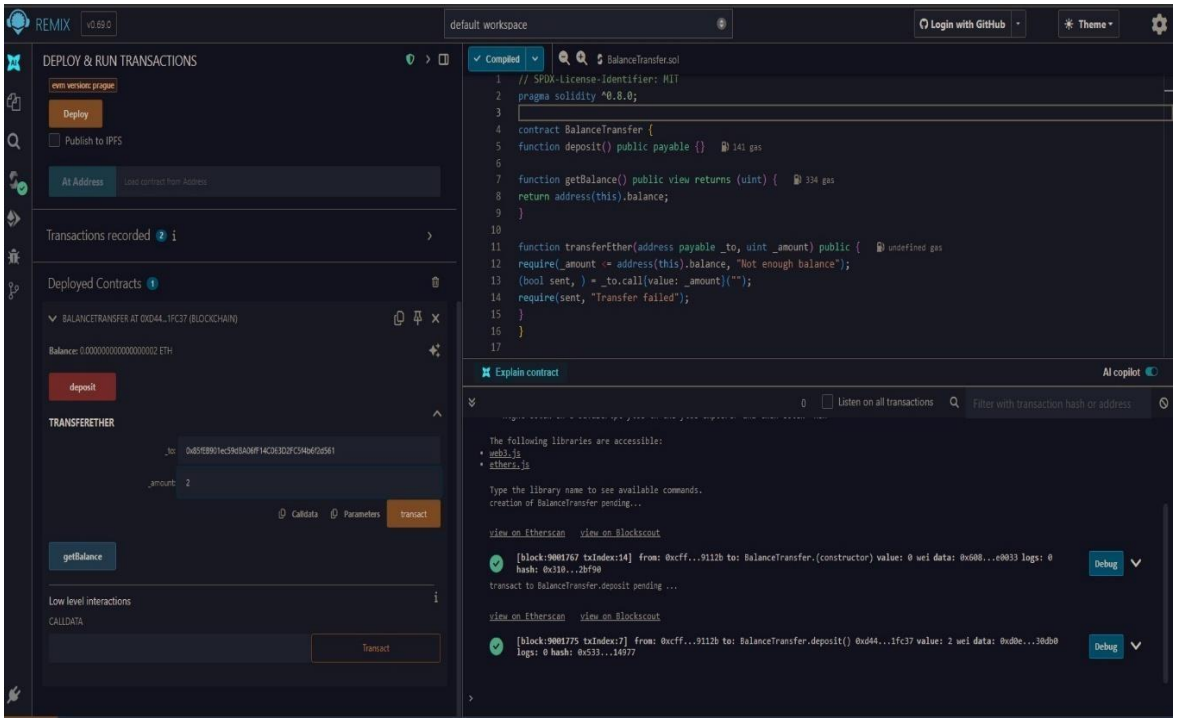
- Click getBalance().
- Output will be 2 (in wei).

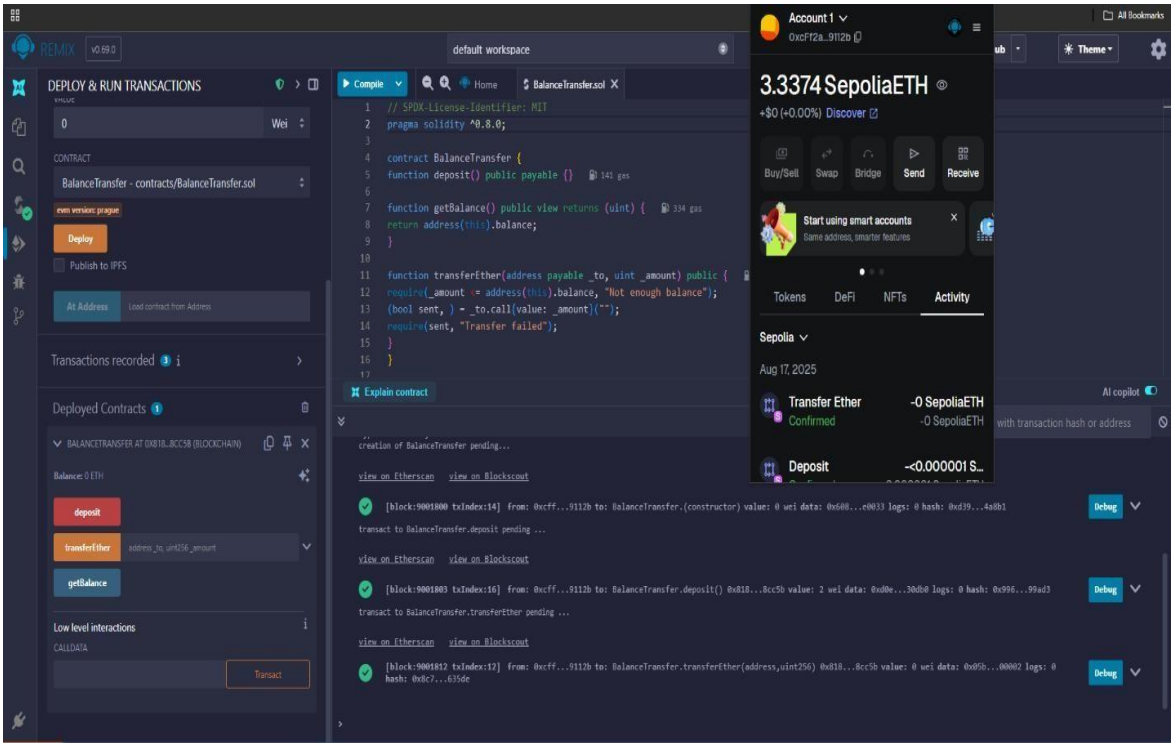
Step 7: Transfer Balance to Another Account

- Copy a recipient Ethereum address.
- In transferEther: o _to → recipient address. o _amount → 2.
- Click transact, confirm in MetaMask.
- Contract balance reduces to 0.

Output:


REMIX
default workspace
DEPLOY & RUN TRANSACTIONS
Injected Provider - MetaMask
Sepolia (11155111) network
Create Smart Account
Deploy
Publish to IPFS
At Address
Transactions recorded
Deployed Contracts
BalanceTransfer.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract BalanceTransfer {
function deposit() public payable {}
function getBalance() public view returns (uint) {
return address(this).balance;
}
function transferEther(address payable _to, uint _amount) public {
require(_amount <= address(this).balance, "Not enough balance");
(bool sent,) = _to.call{value: _amount}("");
require(sent, "Transfer failed");
}
}
Explain contract
Welcome to Remix 0.69.0
MetaMask
Account 1
Sepolia
Deploy a contract
This site wants you to deploy a contract
Estimated changes
No changes
Request from
remix.ethereum.org
Network fee
0.0004 SepoliaETH
Speed
Market ~16 sec
Cancel
Confirm


REMIX
default workspace
Login with GitHub
Theme
DEPLOY & RUN TRANSACTIONS
Deploy
Publish to IPFS
At Address
Transactions recorded
Deployed Contracts
BALANCETRANSFER AT 0XD44...1FC37 (BLOCKCHAIN)
deposit
TRANSFERETHER
getBalance
Low level interactions
CALLDATA
Transact
BalanceTransfer.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract BalanceTransfer {
function deposit() public payable {}
function getBalance() public view returns (uint) {
return address(this).balance;
}
function transferEther(address payable _to, uint _amount) public {
require(_amount <= address(this).balance, "Not enough balance");
(bool sent,) = _to.call{value: _amount}("");
require(sent, "Transfer failed");
}
}
Explain contract
AI copilot
Listen on all transactions
The following libraries are accessible:
web3.js
ethers.js
Type the library name to see available commands.
creation of BalanceTransfer pending...
view on Etherscan
view on Blockscout
[block:9001767 txIndex:14] from: 0xcff...9112b to: BalanceTransfer.(constructor) value: 0 wei data: 0x608...e0033 logs: 0 hash: 0x310...2bf90
Debug
transact to BalanceTransfer.deposit pending ...
[block:9001775 txIndex:7] from: 0xcff...9112b to: BalanceTransfer.deposit() 0xd44...1fc37 value: 2 wei data: 0xd0e...30db0 logs: 0 hash: 0x533...14977



Conclusion

The smart contract was created and tested successfully. It enabled depositing 2 wei, checking balance, and transferring funds, demonstrating basic Ether transactions in Solidity.

Experiment 8

Write a program to perform token Creation and Management on Ethereum

Aim

To create and deploy my own token, EduToken (EDU), on Ethereum and demonstrate transferring tokens between accounts.

Procedure

Step 1: Open Remix IDE

Go to Remix.

Create a new file → SimpleToken.sol

Step 2: Paste the Token Code

```
// SPDX-License-Identifier: MIT pragma solidity
^0.8.24;

contract SimpleToken {
  string public name = "EduToken";
string public symbol = "EDU";
uint8 public decimals = 18;
uint256 public totalSupply;

    mapping(address => uint256) public balanceOf;
  event Transfer(address indexed from, address indexed to, uint256 value);
constructor(uint256 initialSupply) {
 totalSupply = initialSupply * 10 ** uint256(decimals);
  balanceOf[msg.sender] = totalSupply;             emit Transfer(address(0), msg.sender,
totalSupply);
    }
```

```
function transfer(address to, uint256 value) public returns (bool) {
require(balanceOf[msg.sender] >= value, "Not enough tokens");
balanceOf[msg.sender] -= value;
balanceOf[to] += value;
emit Transfer(msg.sender, to, value);
return true;
}
}
```

Step 3: Compile the Contract

- Click the Solidity Compiler tab.
- Select version 0.8.24.
- Click Compile SimpleToken.sol

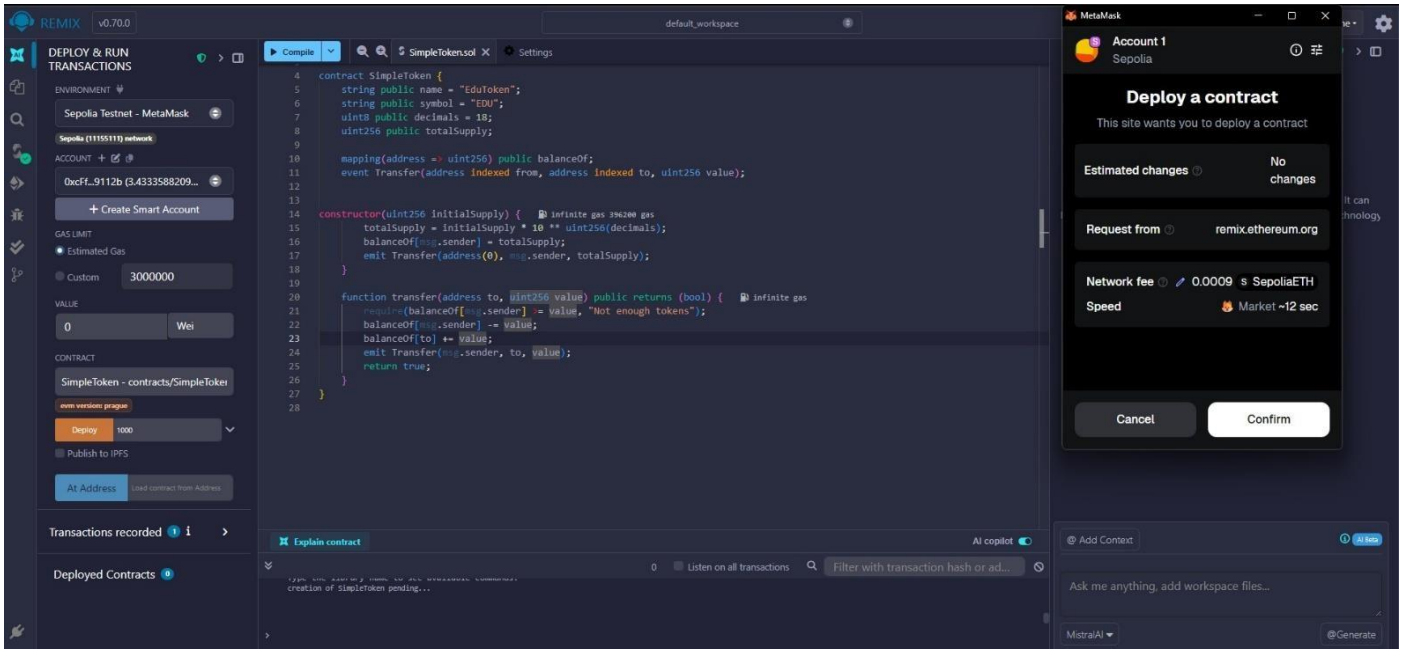
Step 4: Select Environment

- Sepolia Testnet – MetaMask
- Ensure MetaMask is installed and connected to the Sepolia network.
- Add some Sepolia test ETH from a faucet to pay for deployment gas.

Step 5: Deploy the Contract

- In Remix, select SimpleToken contract.
- Enter initial supply (e.g., 1000 tokens).
- Click Deploy → confirm the transaction in MetaMask.

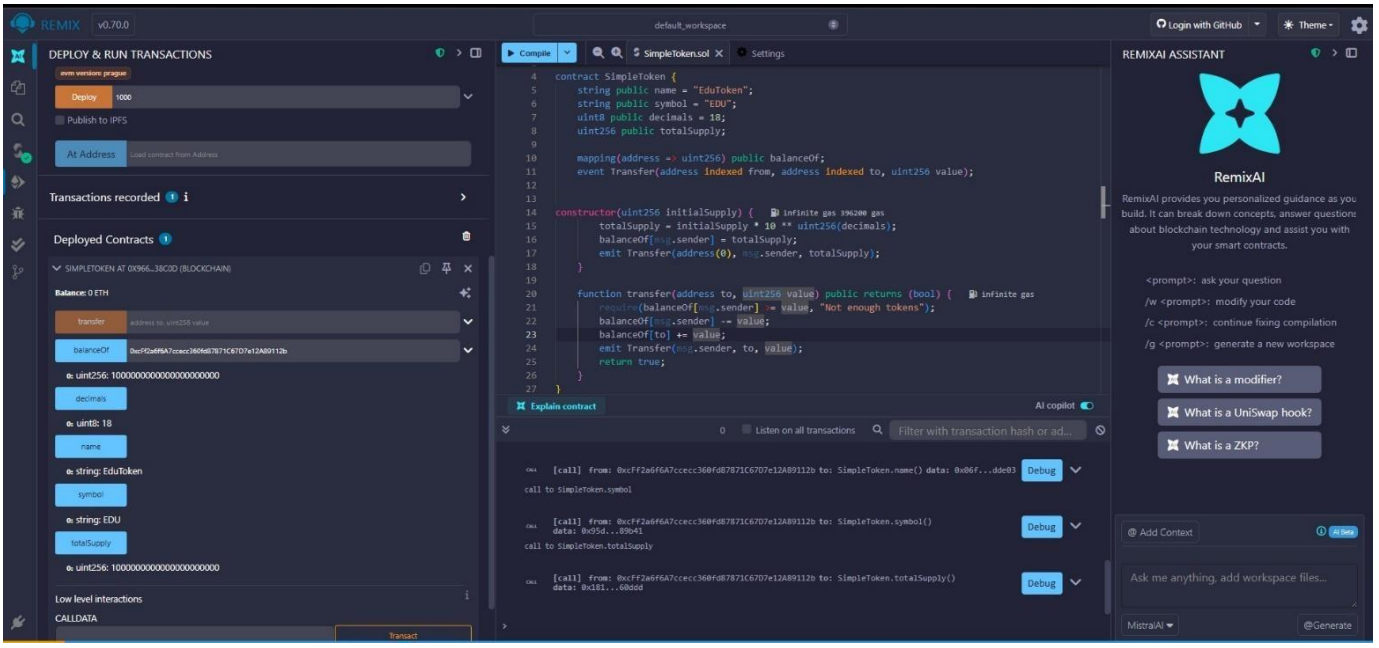
Output:



Step 6: Check Initial Balance

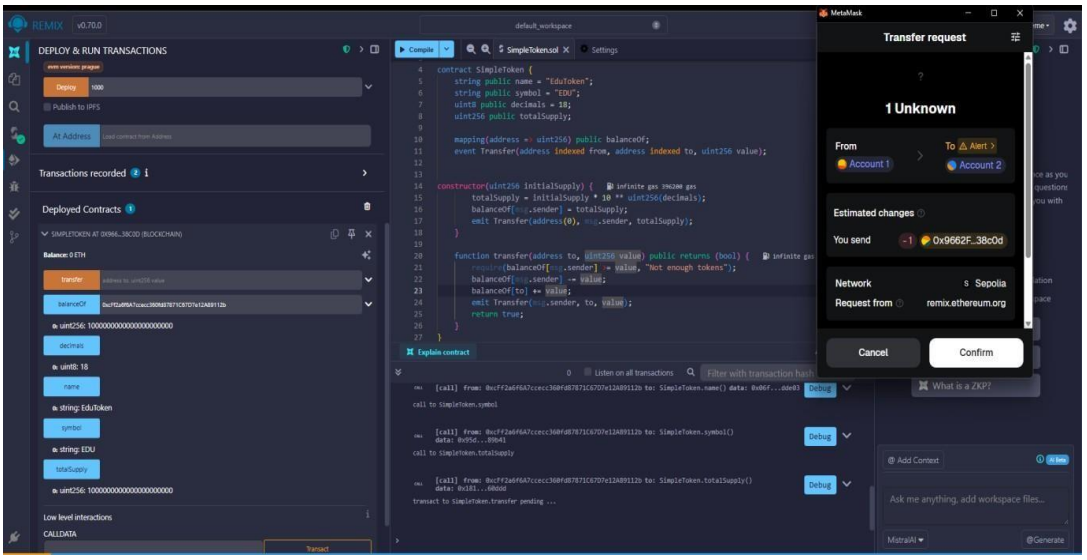
- Use balanceOf(yourAddress) → shows deployer balance in smallest units (1 token = 10^{18} units).
- Example: 1000 tokens = 1000000000000000000000 units.

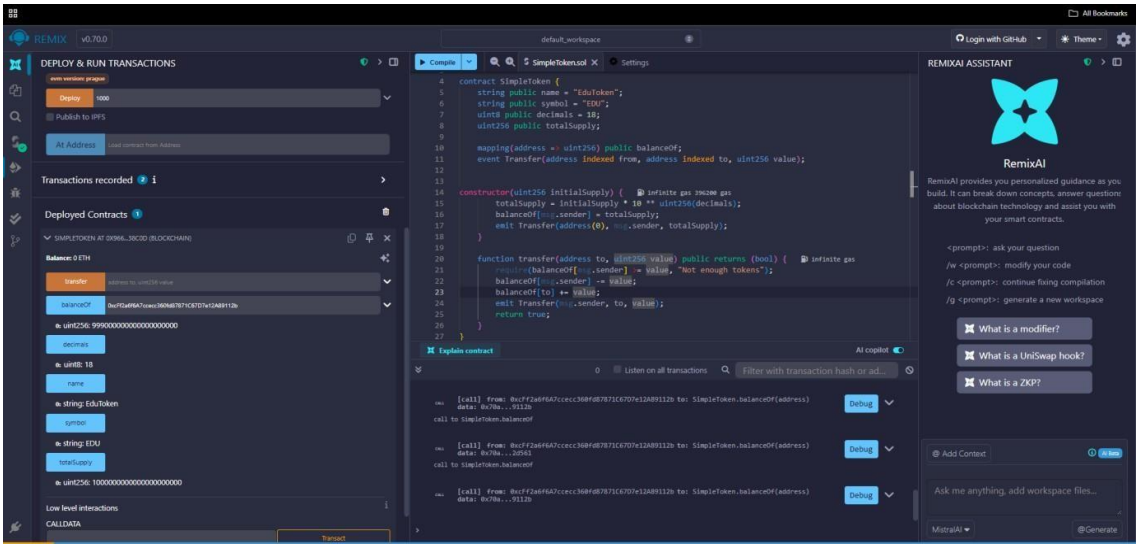
Output:



Step 7: Transfer Tokens

- Pick another account address from MetaMask.
- Use transfer(to, value) to send tokens.
- Example: To send 1 token, enter 1000000000000000000 units.
- Check balances of both sender and receiver to verify the transfer.





Conclusion

The SimpleToken was successfully deployed on the Sepolia testnet. The experiment showed token creation, transfers between accounts, and correct balance updates, demonstrating the basic lifecycle of an ERC-20 token.

Experiment 9

Setup Metamask in the System and Create a wallet in the Metamask with Test Network .

Aim

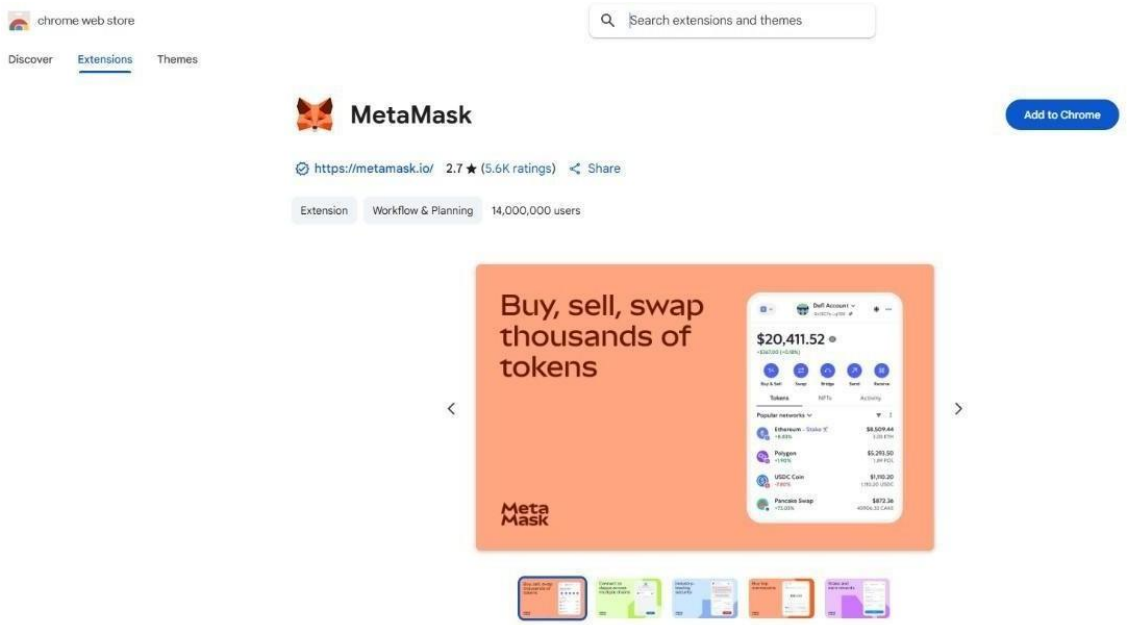
To set up MetaMask in the browser, create a wallet, enable the Sepolia test network, receive Sepolia test ETH from a faucet, and verify the transfer.

Procedure

Step 1: Install MetaMask Extension

- Open your browser (Chrome, Edge, Firefox, or Brave).
- Go to the browser's Extensions / Add-ons page.
    - Example (Chrome): Click the three dots menu → Extensions → Manage Extensions → Open Chrome Web Store.
- In the store, search for MetaMask.
- Click Add to Browser → Add Extension.
- Once installed, pin the MetaMask icon in the browser toolbar for quick access.

Output:



Step 2: Create a New Wallet

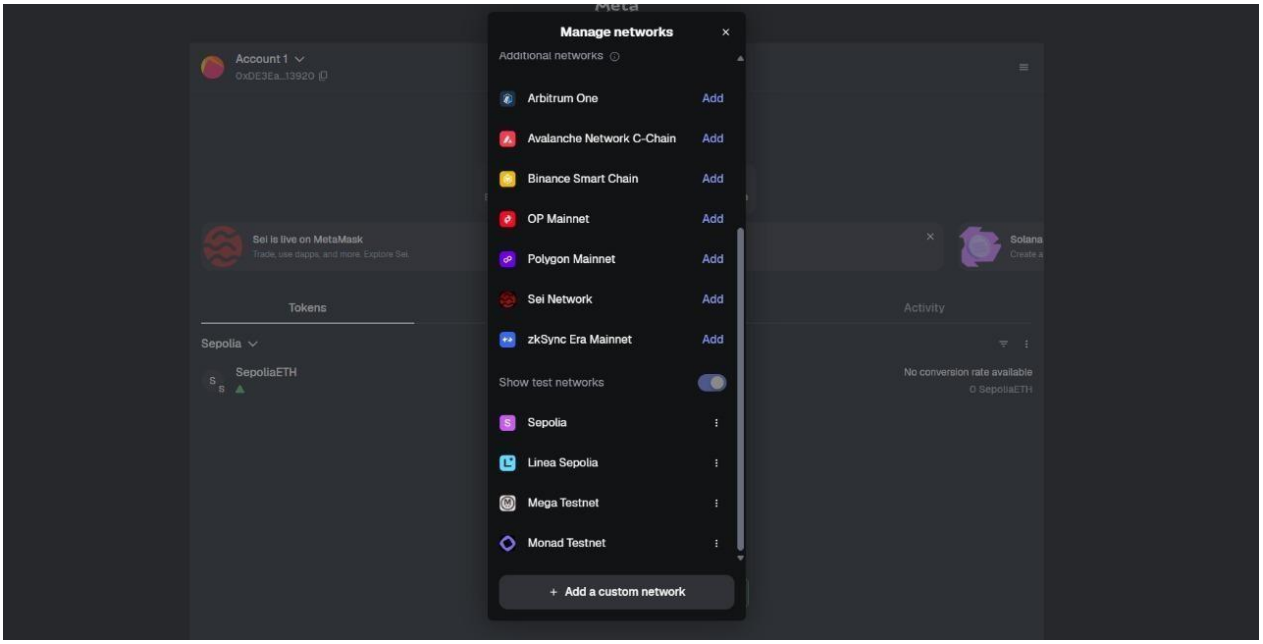
- Click on the MetaMask icon in your browser toolbar.
- Select Get Started → Create a Wallet.
- Set a strong password and store it safely.
- Note down your Secret Recovery Phrase (12 words) on paper — never share it.

- Confirm the phrase in the correct order.

Step 3: Enable Test Networks

- Click the profile icon in MetaMask (top-right).
- Go to Settings → Advanced.
- Turn ON Show test networks.
- Click the network dropdown (top-left) and select Sepolia test network.

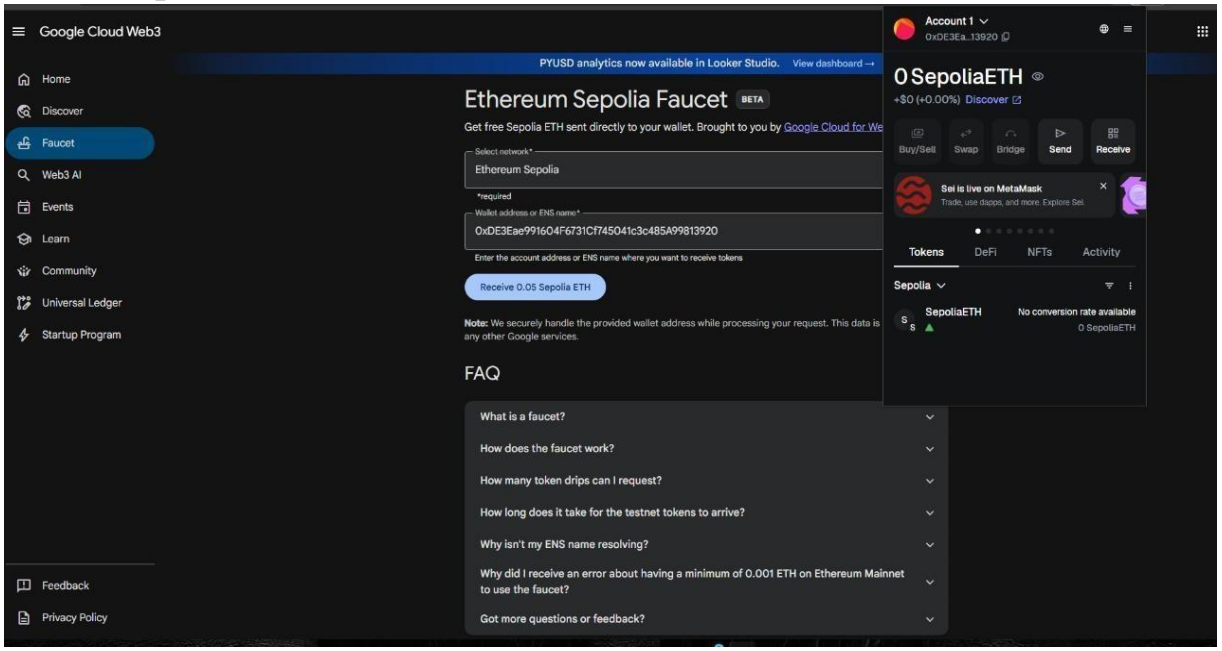
Output:



Step 4: Get Sepolia Test ETH

- Visit Google Cloud Sepolia Faucet
- Paste your wallet address into the faucet form.
- Request Sepolia ETH and wait for it to appear in your wallet (may take seconds to a few minutes).

Output:



Step 5: Verify

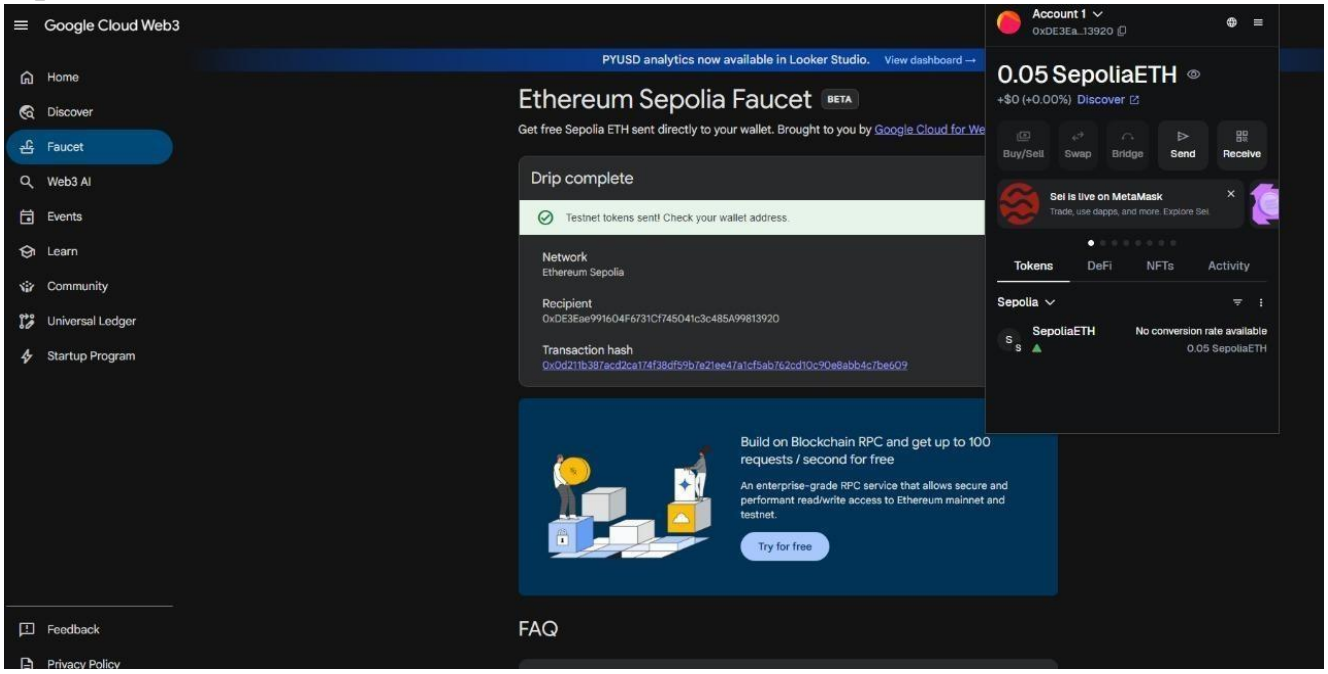
Transaction

- In MetaMask, ensure the network is set to Sepolia.
- Check your balance - the test ETH should be visible.
- Click Activity to view the transaction details.

Step 6 (Optional): Send Test Transaction

- In MetaMask, click Send.
- Enter another Sepolia address and a small ETH amount.
- Click Next → Confirm.
- Verify in the Activity tab that the transaction succeeded.

Output:



Conclusion

In this experiment, MetaMask was installed as a browser extension through the browser's Extensions page, a new wallet was created, the Sepolia test network was enabled, and test ETH was obtained from a faucet. The transaction was successfully verified in MetaMask.

Experiment 10

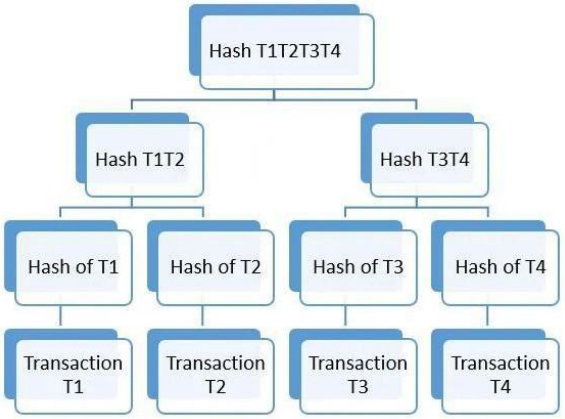
Develop a program to implement blockchain in Merkle Trees.

Aim

To create a simple blockchain in Python using Merkle Trees and demonstrate how blocks are linked using hashes and how transactions are summarized using a Merkle Root.

Merkle Tree Explanation:

A Merkle Tree is a tree-like data structure used in blockchains where transactions are hashed and combined in pairs until a single Merkle Root is formed. This root helps quickly verify transactions without checking them all.



Procedure

Step 1: Open VS Code

• Launch Visual Studio Code on your computer.

Step 2: Create a New Python File

- Click File → New File.
- Save it as merkle_blockchain.py

Step 3: Write the Code

• Copy and paste the following Python code into merkle_blockchain.py:

```
import hashlib
import time
```

```
# Function to calculate SHA-256 hash
def sha256(data):
    return hashlib.sha256(data.encode()).hexdigest()
 # Function to build Merkle Root from transactions
 def merkle_root(transactions):
    hashes = [sha256(tx) for tx in transactions]
while len(hashes) > 1:
  if len(hashes) % 2 != 0:
            hashes.append(hashes[-1])
 new_hashes = []
  for i in range(0, len(hashes), 2):
   new_hash = sha256(hashes[i] + hashes[i+1])
new_hashes.append(new_hash)
  hashes = new_hashes  return hashes[0]
 class Block:
  def __init__(self, index, prev_hash, transactions):
        self.index = index
 self.prev_hash = prev_hash  self.transactions =
transactions          self.merkle_root =
merkle_root(transactions)          self.timestamp =
time.ctime()
 self.hash = self.compute_hash()
    def compute_hash(self):
 header = str(self.index) + self.prev_hash + self.merkle_root + str(self.timestamp)
 return sha256(header) # Blockchain class
class Blockchain:
 def __init__(self):
        self.chain = [self.create_genesis_block()]
 def create_genesis_block(self):
        return Block(0, "0"*64, ["Genesis Block"])
  def get_latest_block(self):
  return self.chain[-1]
def add_block(self, transactions):
        prev_block = self.get_latest_block()
 new_block = Block(len(self.chain), prev_block.hash, transactions)
self.chain.append(new_block)
def display_chain(self):
 for block in self.chain:
           print("\n=== Block", block.index, "===")
print("Previous Hash:", block.prev_hash)
print("Merkle Root :", block.merkle_root)
print("Block Hash  :", block.hash)
 print("Transactions:", block.transactions) my_chain = Blockchain()
my_chain.add_block(["Alice -> Bob: 5", "Bob -> Carol: 2"])
my_chain.add_block(["Eve -> Frank: 3", "George -> Harry: 4"])
my_chain.add_block(["Ivan -> Jack: 7", "Karen -> Leo: 6"])
my_chain.display_chain()
```

Step 4: Save the File

- Press Ctrl + S (or click File → Save).

Step 5: Run the Code

- Open Terminal in VS Code
- Run the program using: python

merkle_blockchain.py

Output:

```
D:\files\BLOCKCHAIN>python merkle_blockchain.py

=== Block 0 ===
Previous Hash: 0000000000000000000000000000000000000000000000000000000000000000
Merkle Root : 89eb0ac031a63d2421cd05a2fbe41f3ea35f5c3712ca839cbf6b85c4ee07b7a3
Block Hash  : a862db6c90faefa8aa49bd2a32bb3e135c1e048ee88c8144d5420b80633eb594
Transactions: ['Genesis Block']

=== Block 1 ===
Previous Hash: a862db6c90faefa8aa49bd2a32bb3e135c1e048ee88c8144d5420b80633eb594
Merkle Root : f6fdf55e436cad2b353403ccea0ece4338f8c0f89e5c08dc308563dd126b1b96
Block Hash  : 715a094121edf72ed6916fecfc1c27e4c2a6bb49d40699949e2cf5cf95cec815
Transactions: ['Alice -> Bob: 5', 'Bob -> Carol: 2']

=== Block 2 ===
Previous Hash: 715a094121edf72ed6916fecfc1c27e4c2a6bb49d40699949e2cf5cf95cec815
Merkle Root : 844a6d3243f251015fafc5dac41ceed879e2373dc27d922f6577c84b22b38866
Block Hash  : 82223791a67d7ebcc0e8021c9d0ada79d14ec8e3096094c52f4153878f19db5e
Transactions: ['Eve -> Frank: 3', 'George -> Harry: 4']

=== Block 3 ===
Previous Hash: 82223791a67d7ebcc0e8021c9d0ada79d14ec8e3096094c52f4153878f19db5e
Merkle Root : 63431e5d26bdc27e31bf8938ae6fb644bf4e1e018e8c17e365711bf71681b59b
Block Hash  : 1ce399382a2c3cf6a75d8a7307b52904649ecee661b5424c5ce7dd77454ccd83
Transactions: ['Ivan -> Jack: 7', 'Karen -> Leo: 6']
```

Conclusion

A blockchain using Merkle Trees was implemented in Python, showing how blocks are linked by hashes and how transaction integrity is ensured through the Merkle Root.

Experiment 11

Create multiple accounts in metamask and perform the balance transfer between the accounts and describe the transaction specification

Aim

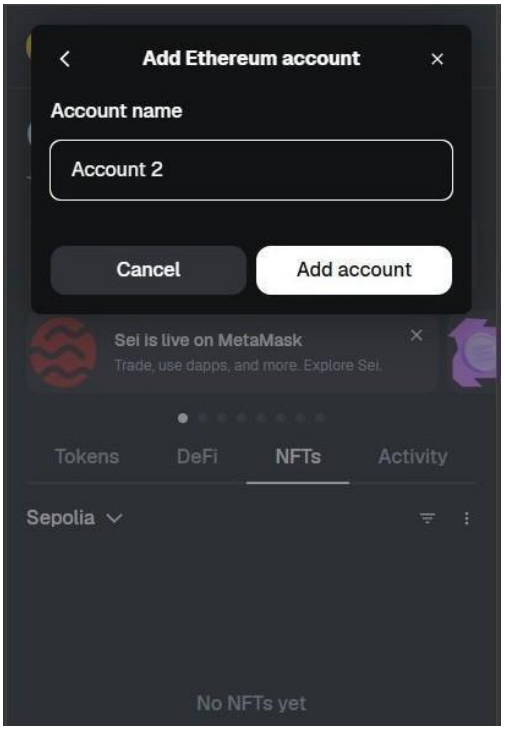
To perform an ETH balance transfer between multiple MetaMask accounts on a test network and record the transaction specifications.

Procedure

Step 1: Create Multiple Accounts

1. In MetaMask, click your account avatar → Create Account.
2. Name it (e.g., Account 1).
3. Repeat to create Account 2.
4. Copy the public addresses for later use.

Output:



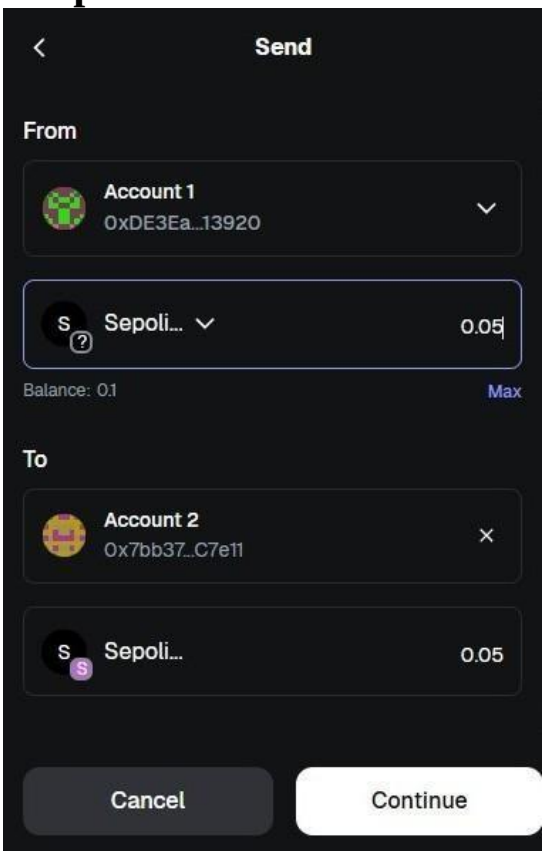
Step 2: Fund the Sender Account

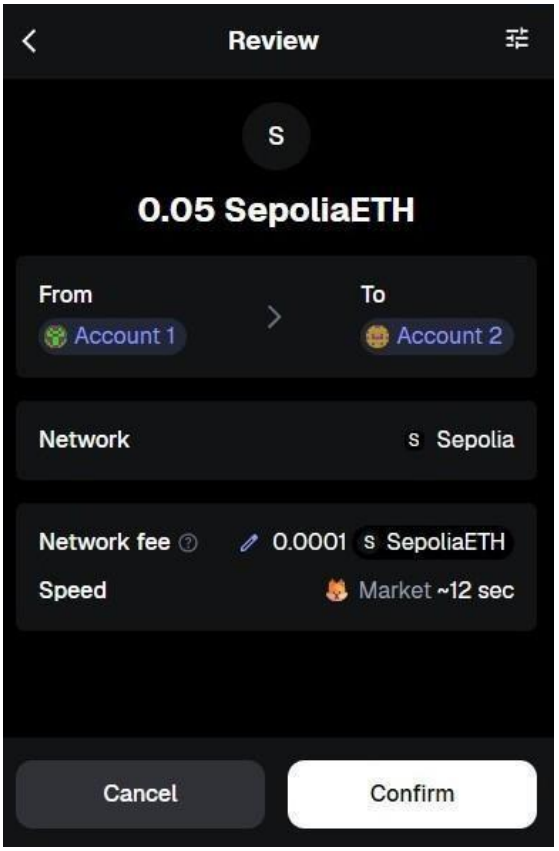
1. Ensure MetaMask is set to Sepolia Test Network.
2. Open a Sepolia faucet and request test ETH for Account 1's address.
3. Wait until the funding transaction is Confirmed in MetaMask.

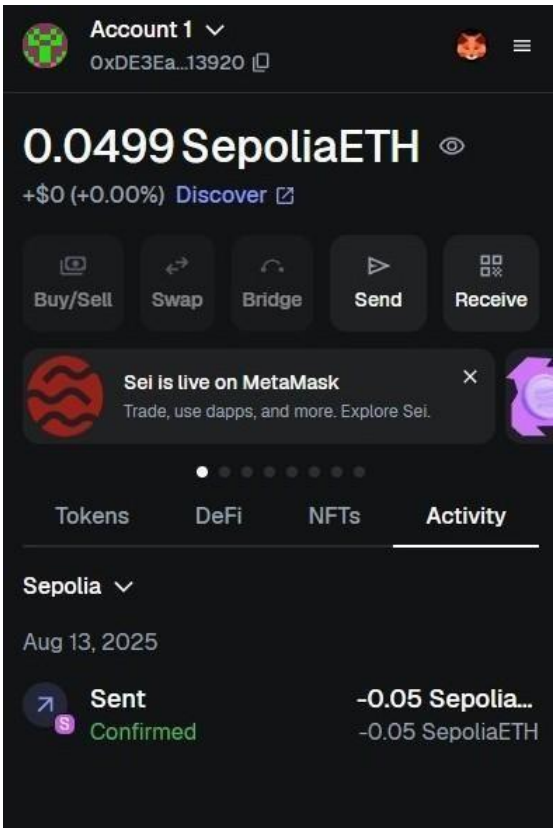
Step 3: Transfer ETH Between Accounts

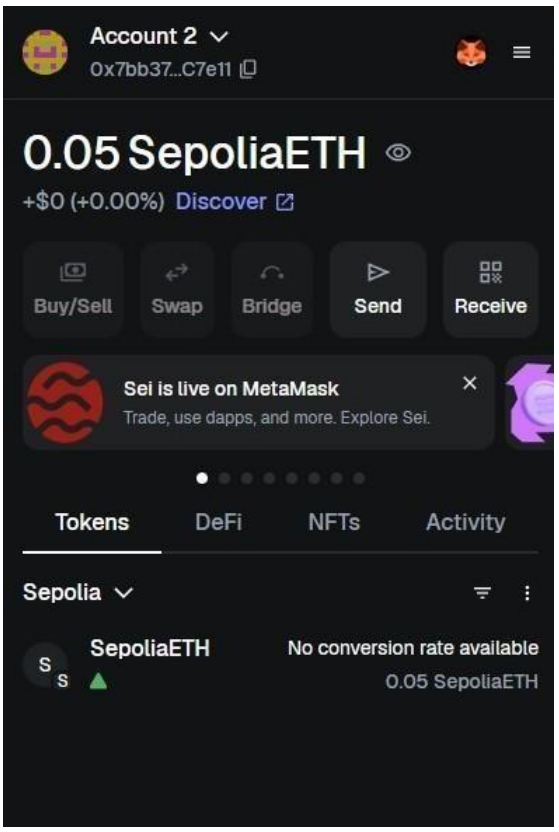
1. In MetaMask, select Account 1 → Send.
2. Paste Account 2's address as the recipient.
3. Enter the amount (e.g., 0.05 ETH).
4. Click Next → review the Estimated Gas Fee → Confirm.
5. Wait for the transaction status to show Confirmed.

Output:





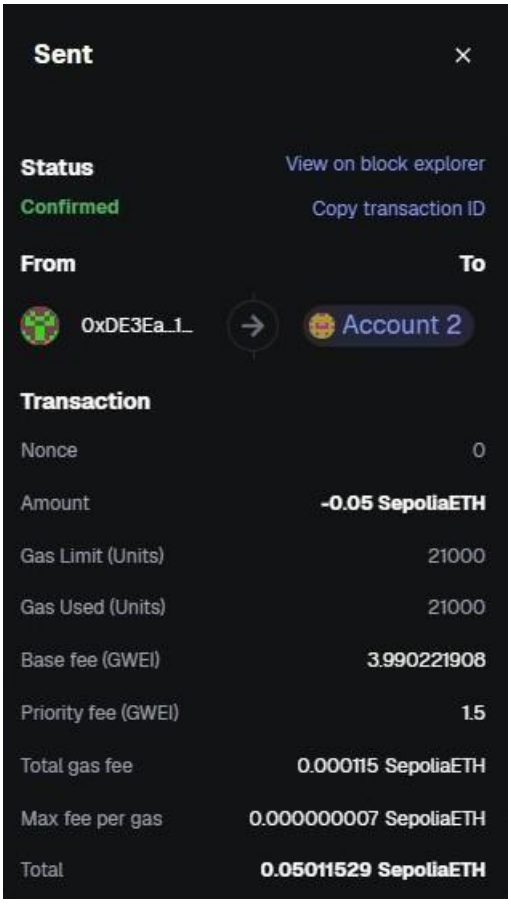




Step 4: View and Record Transaction Specifications

1. In Activity, open the sent transaction → View on Block Explorer.
2. Record these fields for your report: ○ Transaction Hash
   - ○ From (sender) / To (receiver) ○ Value (ETH sent)
   - ○ Gas Price (in Gwei) ○ Gas Limit and Gas Used ○ Max Fee / Max Priority Fee (if shown) ○ Nonce (sender's transaction count) ○ Status (Success/Failed)

Output:



Conclusion

In this experiment, ETH was transferred between two MetaMask accounts on the Sepolia test network. The transaction details from the block explorer helped understand important fields like transaction hash, gas usage, and nonce, showing how Ethereum transactions work.

Experiment 12

Setup the Hyperledger Fabric Network with 2 Organizations 1 Peer Each in the system

Aim:

To set up a Hyperledger Fabric blockchain network consisting of two organizations with one peer each, create a channel, and deploy chaincode to allow interaction with the shared ledger.

Procedure:

Step 1: Prerequisites

Ensure the environment is already set up as done in Experiment 6 (WSL2, Docker, Fabric binaries installed).

Step 2: Start the Network with 2 Organizations

```
cd fabric-samples/test-network

./network.sh up createChannel -c mychannel
```

Step 3: Deploy Chaincode on the Channel

```
./network.sh deployCC -c mychannel -ccn basiccc -ccp ../asset-transfer-basic/chaincodejavascript/ - ccl  javascript
```

Step 4: Initialize Ledger

```
./network.sh cc invoke -c mychannel -ccn basiccc -cci '{"Args":["InitLedger"]}'
```

Step 5: Query Ledger from Org1 / Org2

```
./network.sh cc query -c mychannel -ccn basiccc -cci '{"Args":["GetAllAssets"]}'
```

Conclusion:

A Hyperledger Fabric network with two organizations and one peer each was set up successfully. A channel was created, chaincode deployed, and both organizations accessed the shared ledger.